

PATRONES DE DISEÑO

PATRONES DE DISEÑO

PERO PRIMERO...

- ¿Qué es un Patrón?

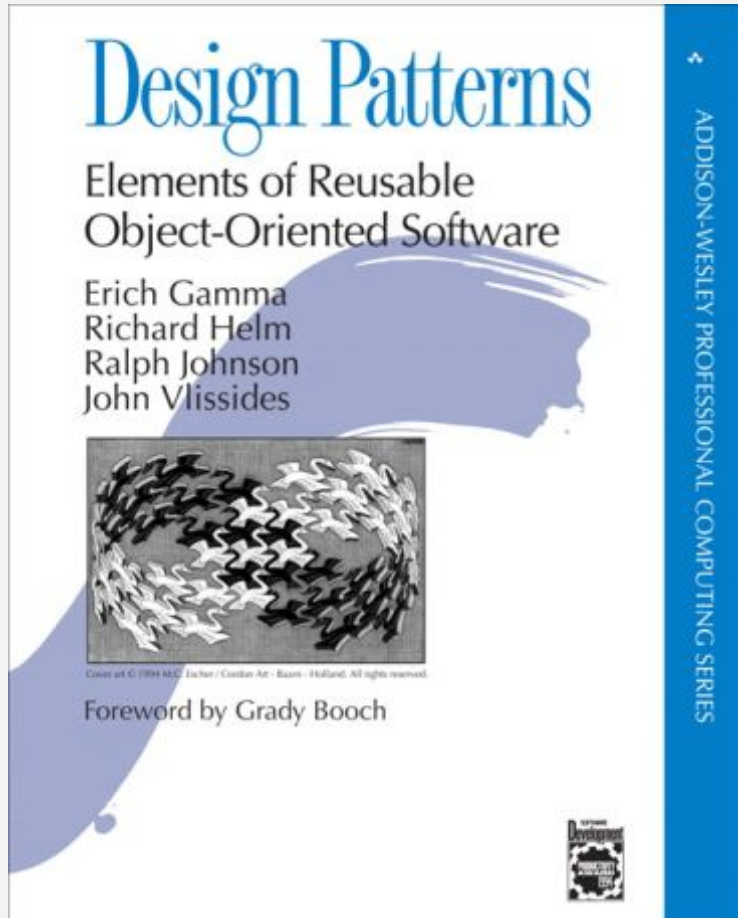
DEFINICIÓN DE PATRÓN

- Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno y describe también el núcleo de la solución al problema, de forma que puede utilizarse un millón de veces sin tener que hacer dos veces lo mismo.
 - Alexander (Arquitecto – Urbanista)

ENTONCES...

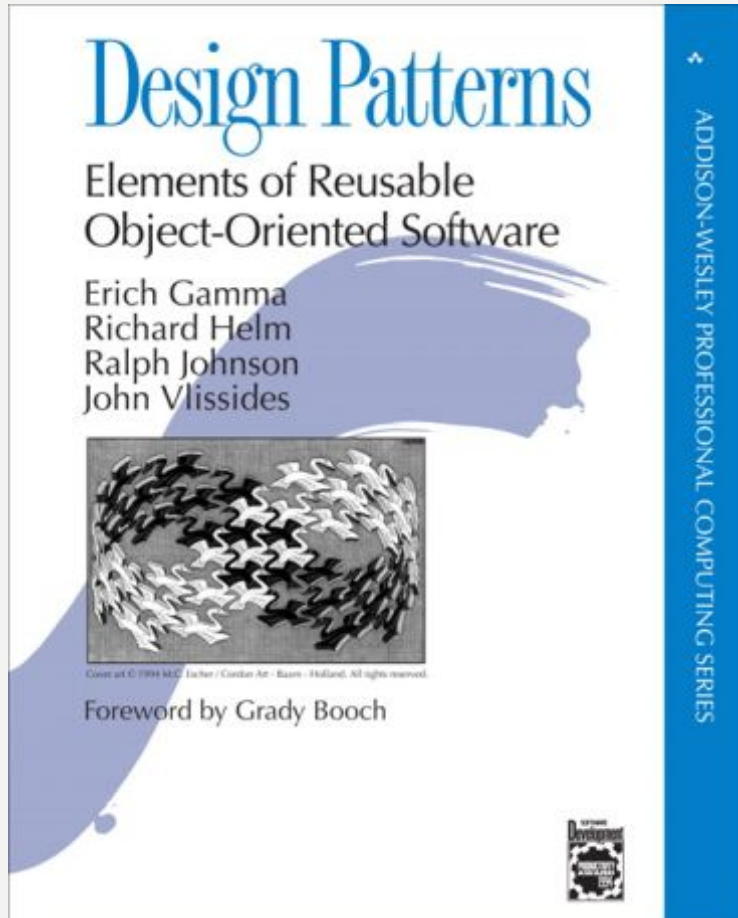
- ¿Qué es un Patrón de Diseño?

¿QUÉ ES UN PATRÓN DE DISEÑO?



- “Es una regla que expresa la relación entre un contexto, un problema y una solución”
- “Es una descripción de clases y objetos comunicándose entre sí adaptada para resolver un problema de diseño general en un contexto particular.” Erich Gamma (miembro del GoF: Gang of Four)

PATRONES DE DISEÑO



- El libro de Gamma define los 23 Patrones que pueden tomarse como fundacionales.
- Si bien estos patrones existían, Gamma y compañía lograron la formalización y aceptación de los mismos.
- Hoy en día hay bastantes más y muchas variaciones de cada uno, pero estudiar los 23 iniciales es el punto de entrada a la utilización de Patrones.

¿CUÁLES SON LOS ELEMENTOS DE UN PATRÓN DE DISEÑO?

- En general, un patrón tendrá 4 elementos claves:
 - Nombre
 - Problema
 - Solución
 - Consecuencias

EL *NOMBRE* DE UN PATRÓN DE DISEÑO

- Nos permite tener un vocabulario en común con otros. Simplifica la comunicación.
- Permite tener un nivel de abstracción más amplio. Con sólo un nombre podemos asociar un problema, solución y consecuencias
- Resumiendo: Con sólo mencionar el nombre de un patrón podremos transmitir un cuerpo de conocimiento amplio y abstracto qué llevaría mucho más tiempo de explicar en detalle cada vez que se lo necesita.

EL ***PROBLEMA*** DE UN PATRÓN DE DISEÑO

- Nos indicará ***cuándo*** utilizar un patrón.
- Nos brinda el contexto del problema a solucionar
- Puede incluir precondiciones, estructuras, algoritmos, etc. que son comunes en el problema a resolver

LA *SOLUCIÓN* DE UN PATRÓN DE DISEÑO

- Describe los elementos que utiliza el diseño, sus relaciones, responsabilidades y colaboraciones
- No es una implementación en particular, sino un template genérico
- La abstracción representada debe adaptarse a nuestro dominio.

***LAS CONSECUENCIAS* DE UN PATRÓN DE DISEÑO**

- Describe los resultados de aplicar un patrón de diseño
- Como todo lo que hacemos en el diseño el impacto estará en la flexibilidad, extensibilidad y portabilidad
- A menudo las consecuencias de un diseño pasan inadvertidas, enumerarlas explícitamente ayudan a tener en cuenta el impacto de un diseño.

BENEFICIOS DE UTILIZAR PATRONES DE DISEÑO

- Diseñar software orientado a objetos no es una tarea sencilla
- Los diseños deben ser específicos del problema a resolver, pero también lo suficientemente generales para afrontar problemas futuros
- Siempre se debe intentar reutilizar diseños previos, nunca afrontar un problema por primera vez mas de una vez

BENEFICIOS DE UTILIZAR PATRONES DE DISEÑO

- Si bien aun no existe (y probablemente nunca exista) una secuencia mecánica de pasos que me brinden un buen diseño, los patrones son una buena formalización de diseños probados y útiles en infinidad de ocasiones
- Aunque finalmente no exista un patrón para el problema a resolver, siempre son un buen punto de inicio e inspiración.

¿POR QUÉ UTILIZAR PATRONES DE DISEÑO?

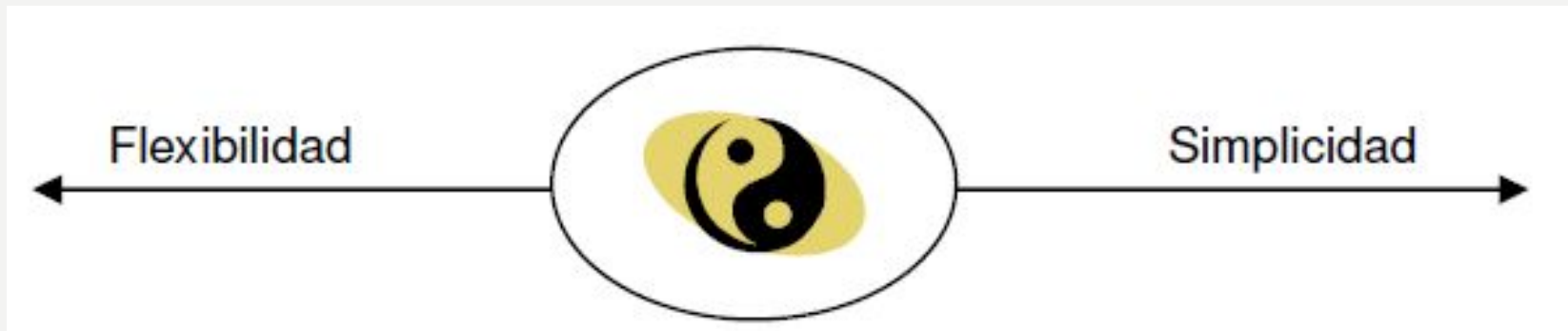
- Porque abre la mente, nos da la capacidad de explorar mejoras a nivel de diseño
- Porque al adaptarnos al vocabulario de la comunidad ganamos tiempo
- Nos obliga a ir adelante y atrás en el diseño y no pensar exclusivamente en una solución de compromiso para hoy.
 - Esto no sólo produce diseños más elegantes, sino que así también aprendo a equilibrar hasta cuándo dejo la puerta abierta para cambios y hasta cuándo cierro una solución para poder implementarla
- Es una forma de documentación que fomenta la reutilización de nuestros diseños y nos permite compartirlos de forma mas sencilla y formal.

CONSEJOS PARA APLICARLOS

- Intent, Motivation y Applicability
 - Cada patrón posee estas tres secciones que nos permitirán definir si un patrón puede “calzar” con el problema que estoy intentando resolver.
- Categorías de Patrones
 - Creacionales: abstraen el proceso de instanciación
 - Estructurales: define estructuras de clases y objetos
 - De comportamiento: se ocupan de la asignación de responsabilidades entre objetos y como se comunican entre sí
- No todo puede ser variable, Gamma recomienda concentrarse en *encapsular la parte más variable* de nuestra aplicación

CÓMO NO USARLOS

- Cada vez que decidimos flexibilizar una parte de nuestro sistema, estamos agregando complejidad al diseño y esto redundará en un mayor costo de implementación. Aparecen fuerzas contrapuestas:



¿EN QUÉ ETAPAS APLICARLOS?

- *Análisis, en esta etapa todavía estamos aprendiendo el dominio por lo cual es imposible aplicar patrones*
- Diseño, en esta etapa aún no tenemos conocimiento completo pero con ayuda del usuario experto se puede identificar qué es lo que puede cambiar en el sistema.
- Desarrollo, si al desarrollar detectamos algún patrón no diseñado entonces debemos actualizar el diseño.

PUESTA EN COMÚN DE CONCEPTOS NECESARIOS

- Clase abstracta:
 - Clase que no puede ser instanciada y debe ser heredada. Dicta estructura y comportamiento.
- Herencia:
 - Facilita la extensión de las funcionalidades de clases existentes, sin tener que introducir modificaciones en ellas.
- Interfaz:
 - Contiene firmas de métodos y constantes que sirven como el contrato a implementar en las clases que la realizan. Desde el punto de vista de las clases cliente, abstraen un comportamiento común.
- Polimorfismo:
 - Se refiere a la capacidad de que varias clases con un comportamiento común puedan responder de forma diferente a un mismo método.
- Sobrecarga :
 - Posibilidad de tener dos o más métodos con el mismo nombre pero funcionalidad diferente. Diferente firma.

CLASIFICACIÓN DE PATRONES DE DISEÑO

- De creación
 - Solucionan problemas de instanciación de objetos.
- De estructura
 - Se enfocan en la forma estructural de las clases.
- De comportamiento
 - Se centran en la resolución de problemas relacionados con el comportamiento de una aplicación en tiempo de ejecución.

SINGLETON

SINGLETON (CREACIONAL)

- Patrón de diseño creacional
- Intención:
 - Asegura que una clase tiene una única instancia y brinda un punto global de acceso para obtenerla.

Singleton
-instance : Singleton
-Singleton() +Instance() : Singleton

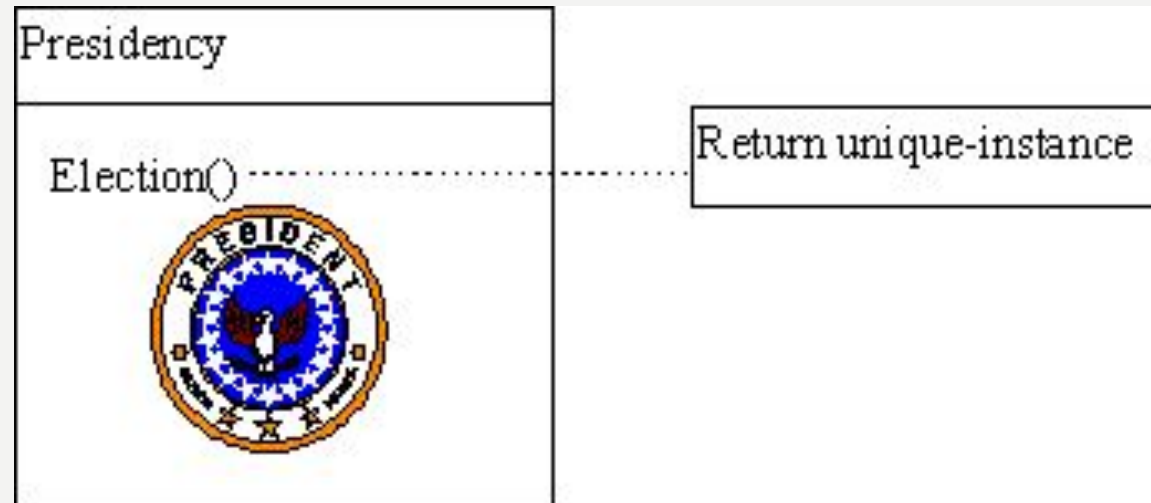
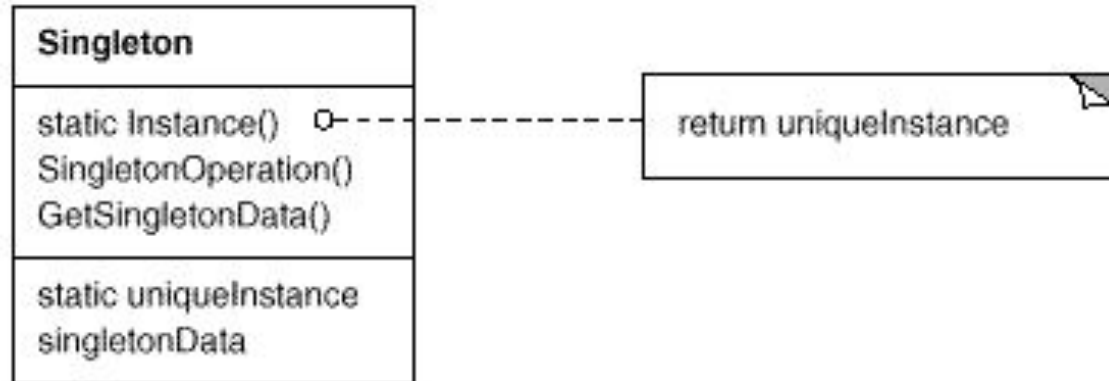
SINGLETON

- Participantes:
 - **Singleton (LoadBalancer)** Define un método **Instance** que permite a los clientes acceder a su única instancia.
 - Instance es un método de clase responsable de crear y mantener su única instancia.
- Colaboraciones
 - Los clientes acceden a la única instancia de Singleton a través del método estático provisto

SINGLETON

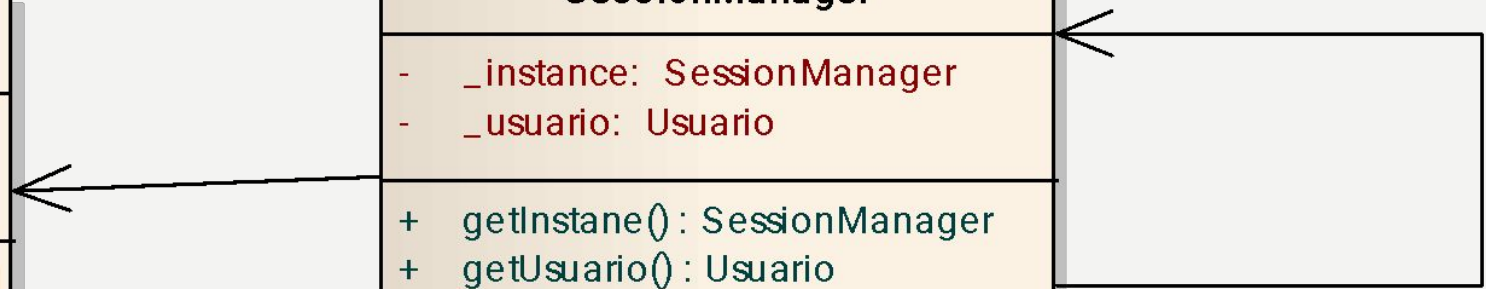
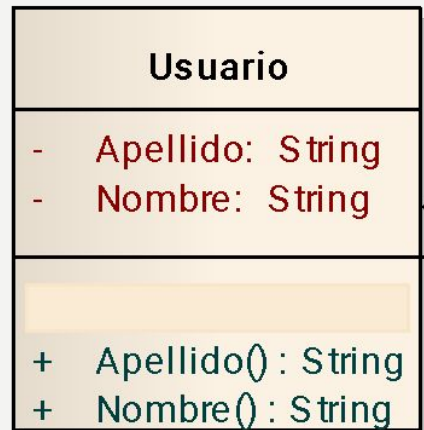
- Consecuencias:
 - Ventaja sobre variables globales. Al utilizar una instancia en lugar de variables globales evitamos tener variables repartidas por doquier y pisar sus nombres en un mismo namespace.
 - Su configuración para una aplicación es sencilla, solo debe utilizarse en run-time
 - Otra forma de lograr la misma funcionalidad es utilizando métodos de clases (static). Pero los métodos estáticos no permiten ser virtual con lo cual no podemos hacer uso de la redefinición y polimorfismo.

SINGLETON



SINGLETON

class Singleton

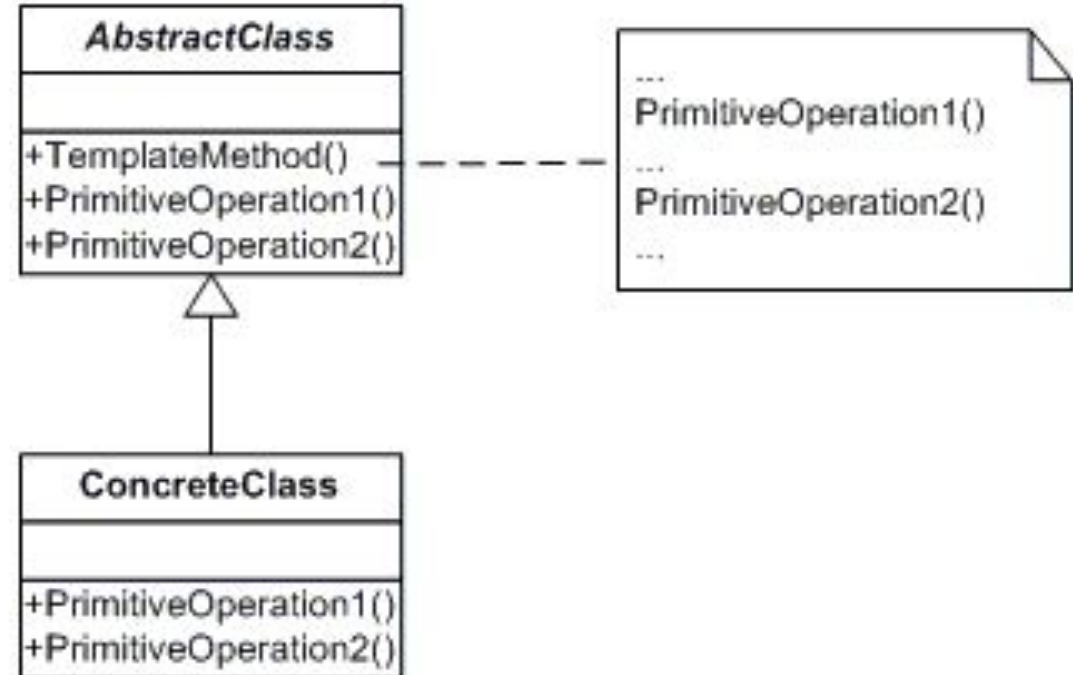




TEMPLATE METHOD

TEMPLATE METHOD (COMPORTAMIENTO)

- Intención:
 - Definir el esqueleto de un algoritmo en una operación determinada. Template Method deja para las subclases la redefinición de ciertos pasos de un algoritmo sin modificar su estructura.



TEMPLATE METHOD

- Participantes:
 - **AbstractClass** (Application)
 - Define **operaciones primitivas** abstractas que las subclases concretas definirán para implementar los pasos de un algoritmo.
 - Implementa un template method (o molde) definiendo el esqueleto de un algoritmo. El template method llamará a las operaciones primitivas así como a operaciones definidas en AbstractClass y otros objetos para realizar el algoritmo completo.
 - **ConcreteClass** (MyApplication)
 - Implementa las operaciones primitivas para realizar los pasos específicos del algoritmo.

TEMPLATE METHOD

- Colaboraciones:
 - ConcreteClass deja los pasos invariantes del algoritmo para que los realice AbstractClass por medio de su template method.
- Aplicaciones (usar cuándo)
 - Para implementar pasos invariantes una sola vez y dejar a las subclasses la implementación de lo que sí puede variar.
 - Cuando el comportamiento común entre varias subclasses debe ser factorizado y puesto en una clase común para evitar código duplicado.
 - Para controlar los puntos de extensión. Se pueden definir operaciones hook que son llamadas por el template method y por lo tanto dejar sólo esos hook como puntos de extensión.

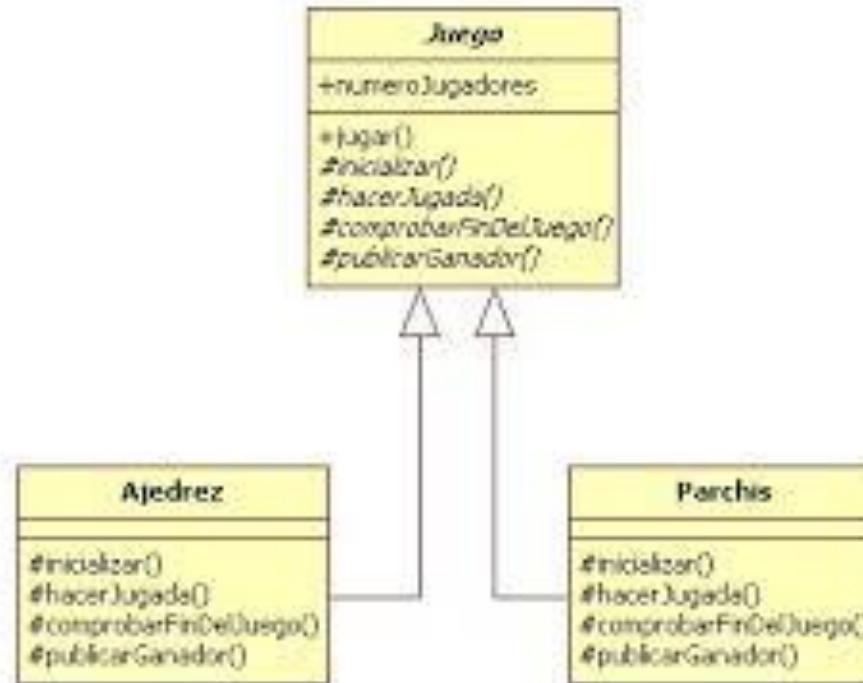
TEMPLATE METHOD

- Consecuencias:
 - Los Template Methods son una de las principales técnicas para reutilización de código y son particularmente importante en las librerías.
- Los template methods llaman a los siguientes tipos de operaciones
 - Operaciones concretas (en ConcreteClass o en el Client).
 - Operaciones concretas de AbstractClass, operaciones útiles para todas las subclases.
 - Operaciones primitivas, las operaciones abstractas
 - Factory Methods (es otro patrón)
 - Hook operations, proveen el comportamiento común a todas las subclases que luego lo redefinen si lo necesitan. Suele no hacer nada.

TEMPLATE METHOD

- Consecuencias:
 - Es importante especificar que métodos son hook (pueden ser redefinidos o no) y cuales son operaciones abstractas (y deben implementarse en la sublcase). En .Net la diferencia la lograremos usando **abstract** en el método que corresponda.

TEMPLATE METHOD

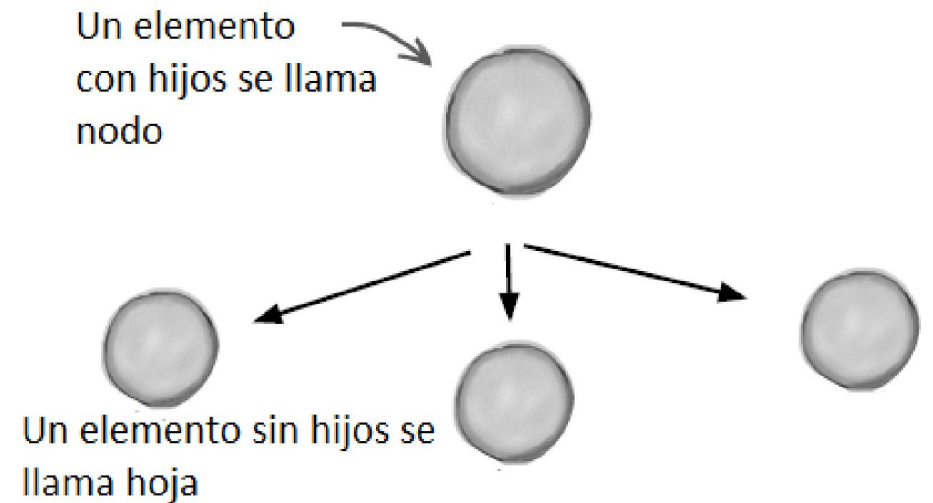




COMPOSITE

COMPOSITE (ESTRUCTURAL)

- Propósito
 - Compone objetos en estructuras de árbol para representar jerarquías de parte-todo.
 - Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.

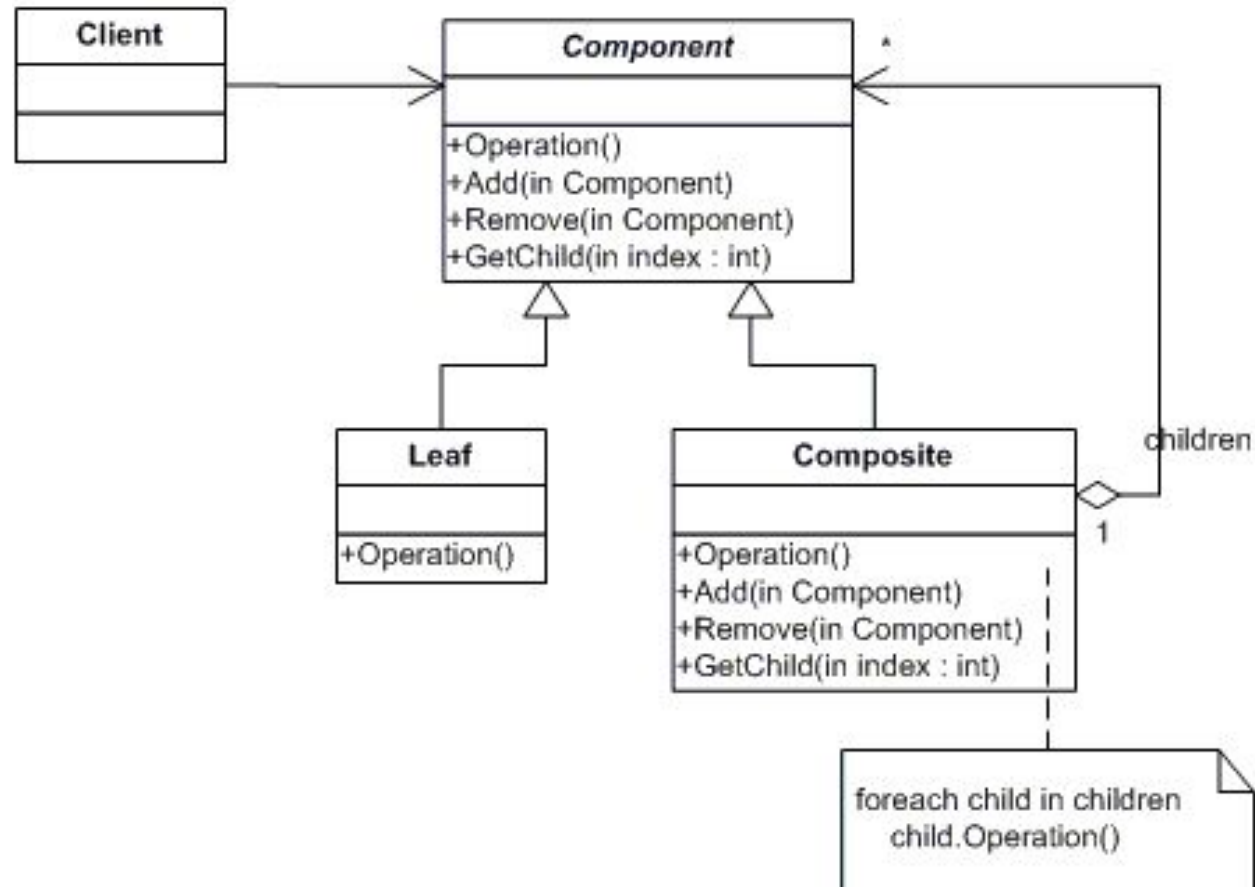


COMPOSITE

- Aplicabilidad
 - Quiera representar jerarquías de objetos todo-parte.
 - Quiere que los clientes sean capaces de obviar las diferencias entre composiciones de objetos y los objetos individuales. Los clientes tratarán a todos los objetos de la estructura compuesta de manera uniforme.

Contenedores que contienen elementos, cada uno de los cuales podría ser un contenedor

COMPOSITE



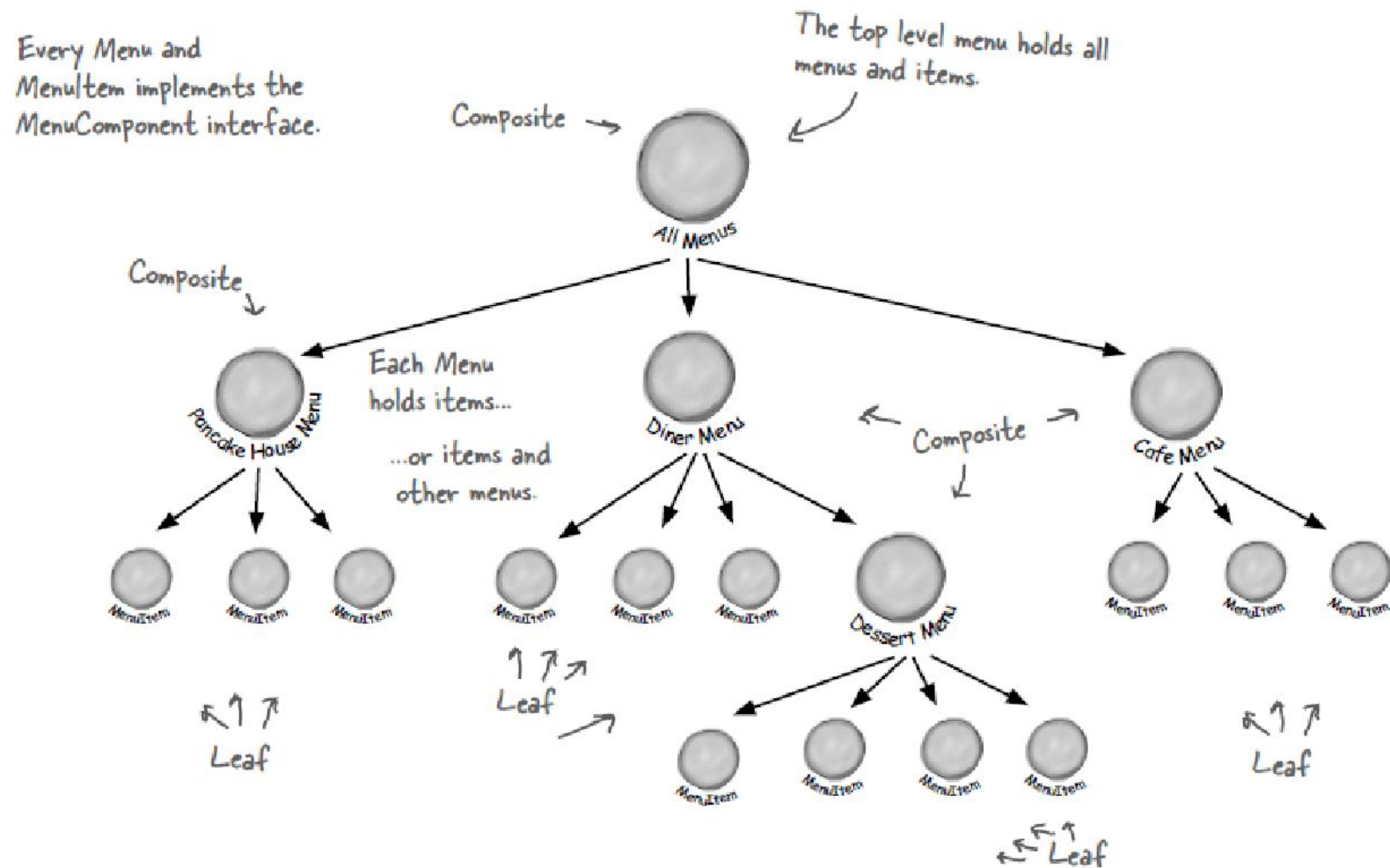
COMPOSITE

- Consecuencias:
 - Define jerarquías de clases formadas por objetos primitivos y compuestos.
 - Simplifica al cliente. El cliente trata las estructuras uniformemente.
 - Facilita añadir nuevos tipos de componentes.
 - Puede hacer que un diseño sea demasiado general.

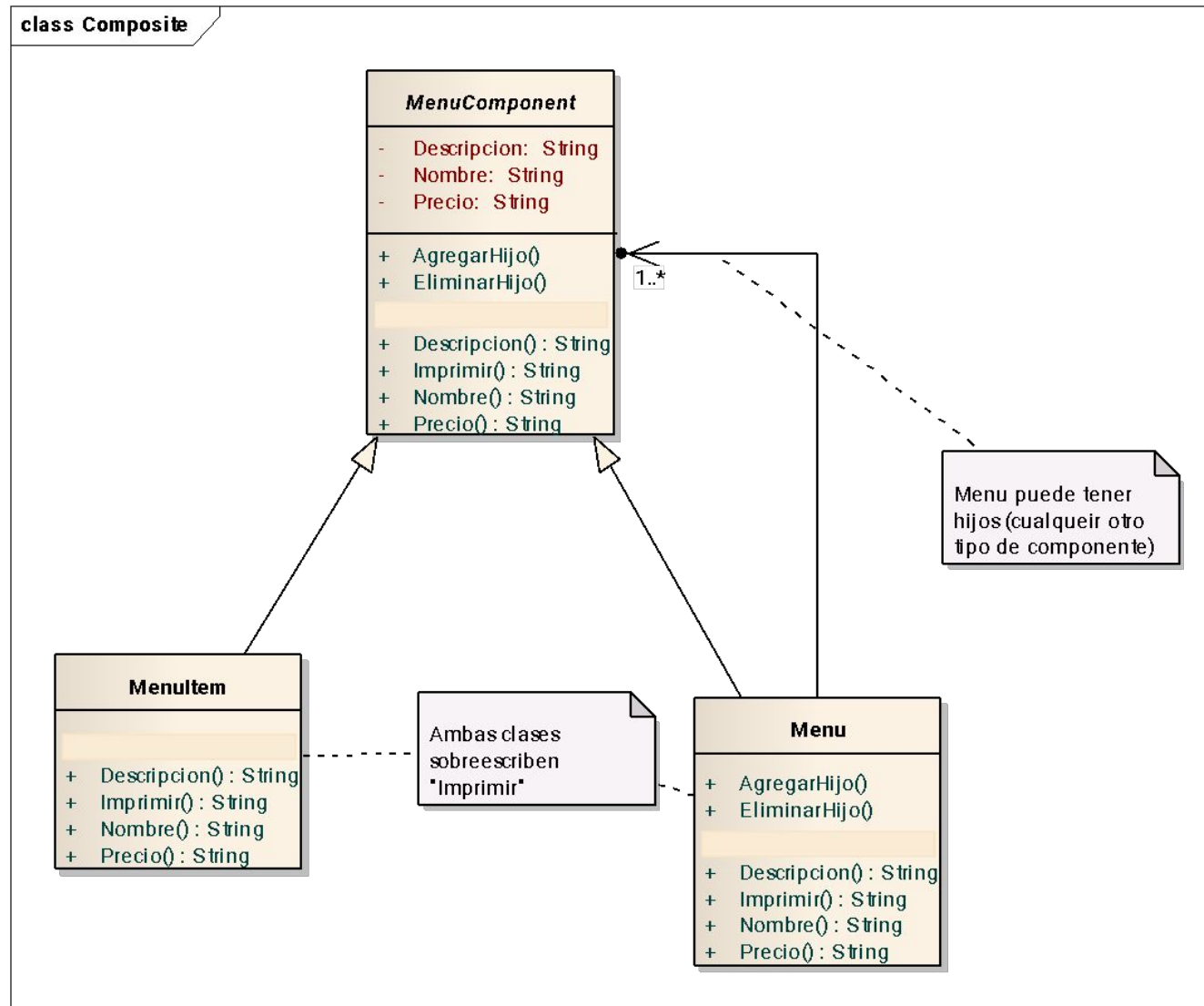
COMPOSITE - EJEMPLO

- Nos piden desarrollar un sistema para manejar los diferentes menús en un restaurante, ya que dependiendo el momento del día el mismo cambia completamente.

COMPOSITE - EJEMPLO

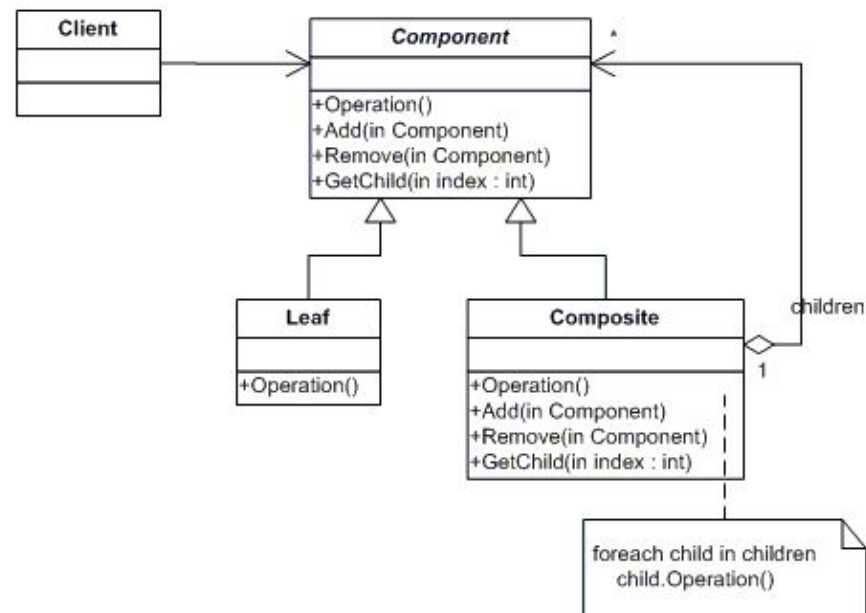


COMPOSITE - EJEMPLO



COMPOSITE (NOTAS)

- Crear una interfaz que oficie de "mínimo denominador común" que haga que los contenedores y los contenidos puedan ser intercambiables
- Todas las clases Contenedor y Contenido implementan la interfaz.
- Métodos de manejo de hijos (Ej. *AddChild()*, *RemoveChild()*) deberían normalmente ser definidos en la clase *Composite*. Por desgracia, el deseo de tratar a los objetos *Leaf* y *Composite* de manera uniforme requiere que dichos métodos sean movidos a la clase abstracta *Component*.
- Las clases Contenedor aprovechan el polimorfismo para delegar en sus objetos Contenido.



A dark brown wavy line runs vertically along the left edge of the slide.A solid yellow vertical bar runs along the right edge of the slide.

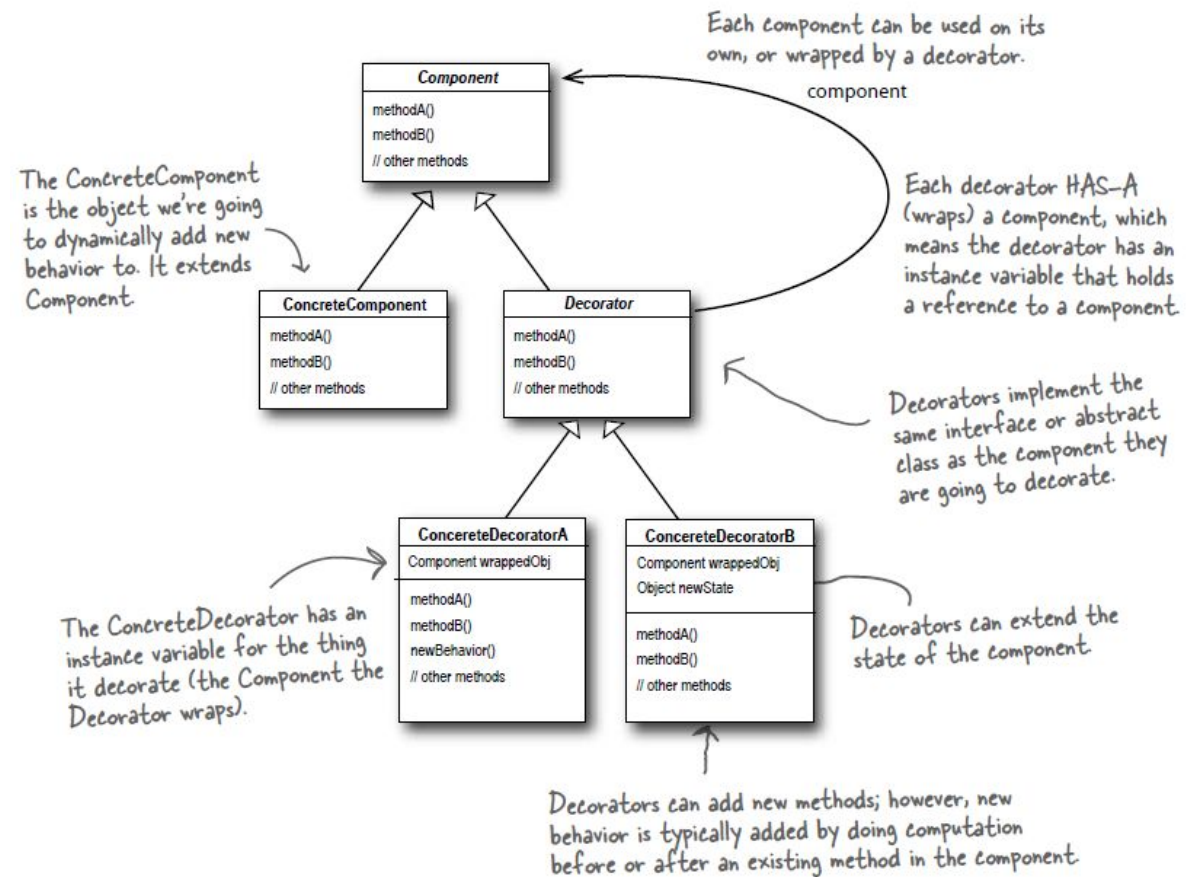
DECORATOR

DECORATOR (ESTRUCTURAL)

- Propósito
 - Asigna responsabilidades adicionales a un objeto dinámicamente, proporcionando una alternativa flexible a la herencia para extender la funcionalidad.
- También conocido como
 - Wrapper (Envoltorio)

DECORATOR

- Estructura



DECORATOR

- Consecuencias:
 - Más flexibilidad que la herencia estática, las responsabilidades se añaden dinámicamente.
 - Evita clases cargadas de funciones en la parte de arriba de la jerarquía.
 - Un decorador y su componente no son idénticos, evitar apoyarse en la identidad de objetos.
 - Muchos objetos pequeños.

DECORATOR - EJEMPLO

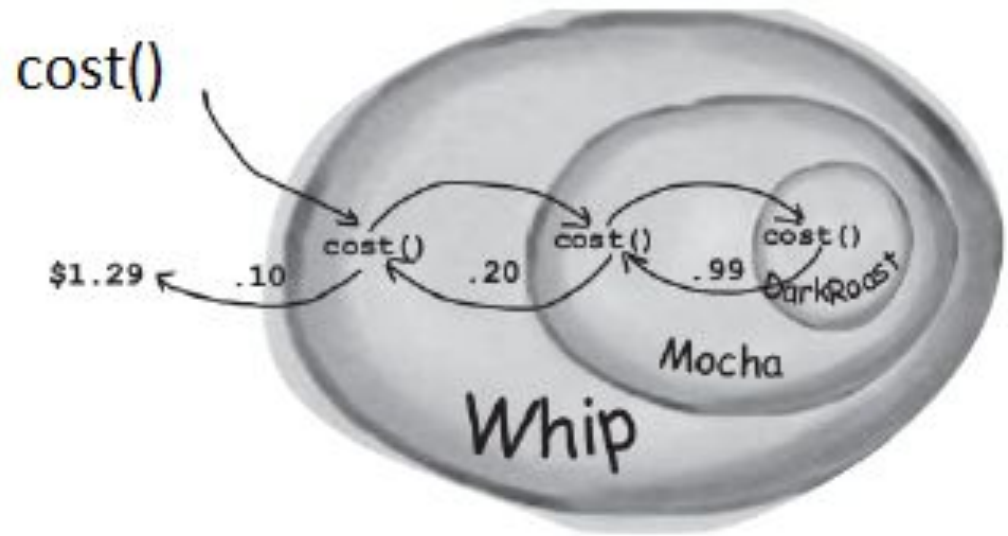
- Nos piden desarrollar un sistema para calcular el costo de una bebida, en base a distintos parámetros:
 - Puede ser café solo (\$10), café expreso (\$12), café descafeinado (\$15) y té tradicional (\$8)
 - Pueden añadirse diferentes condimentos como: leche (\$2), crema (\$4), edulcorante (\$1) y azúcar (\$0.50).
 - Cada bebida o condimento tiene un costo específico, y la combinación da el costo total.

DECORATOR - EJEMPLO

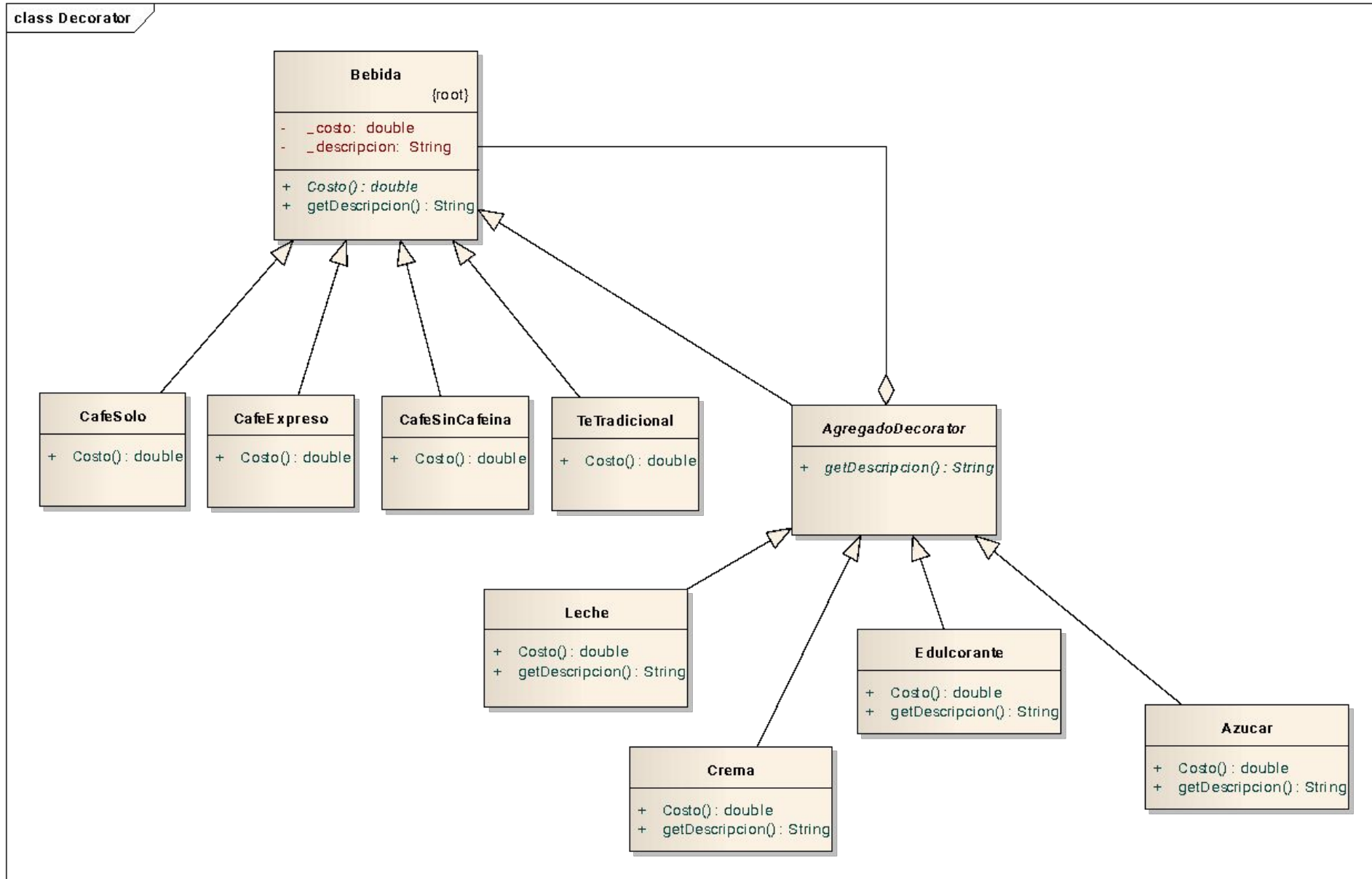
- Pedimos un café
- Le agregamos crema
- Le agregamos azúcar
- Calcular el Costo

DECORATOR - EJEMPLO

- Pedimos un café
- Le agregamos crema
- Le agregamos azúcar
- Calcular el Costo



DECORATOR - EJEMPLO



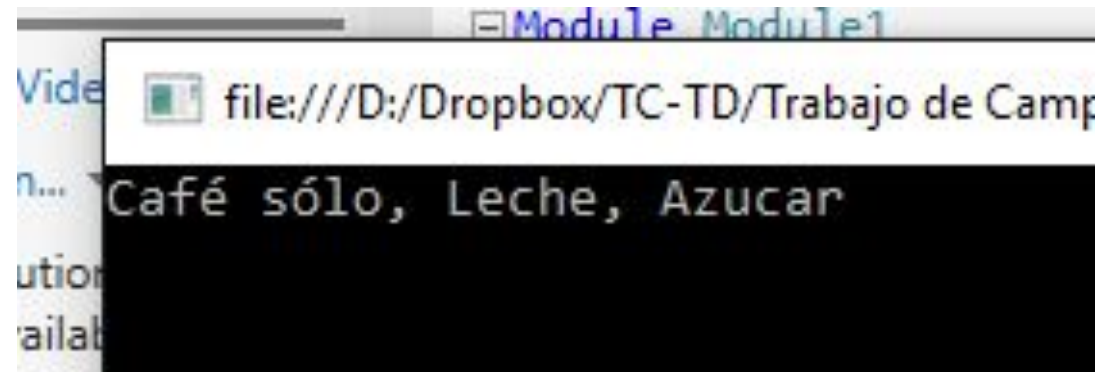
DECORATOR – EJEMPLO EN VB.NET

```
Dim cafecito As Bebida

cafecito = New CafeSolo()
cafecito = New Leche(cafecito)
cafecito = New Azucar(cafecito)

Console.WriteLine(cafecito.GetDescripcion)

Console.ReadKey()
```



DECORATOR Y COMPOSITE

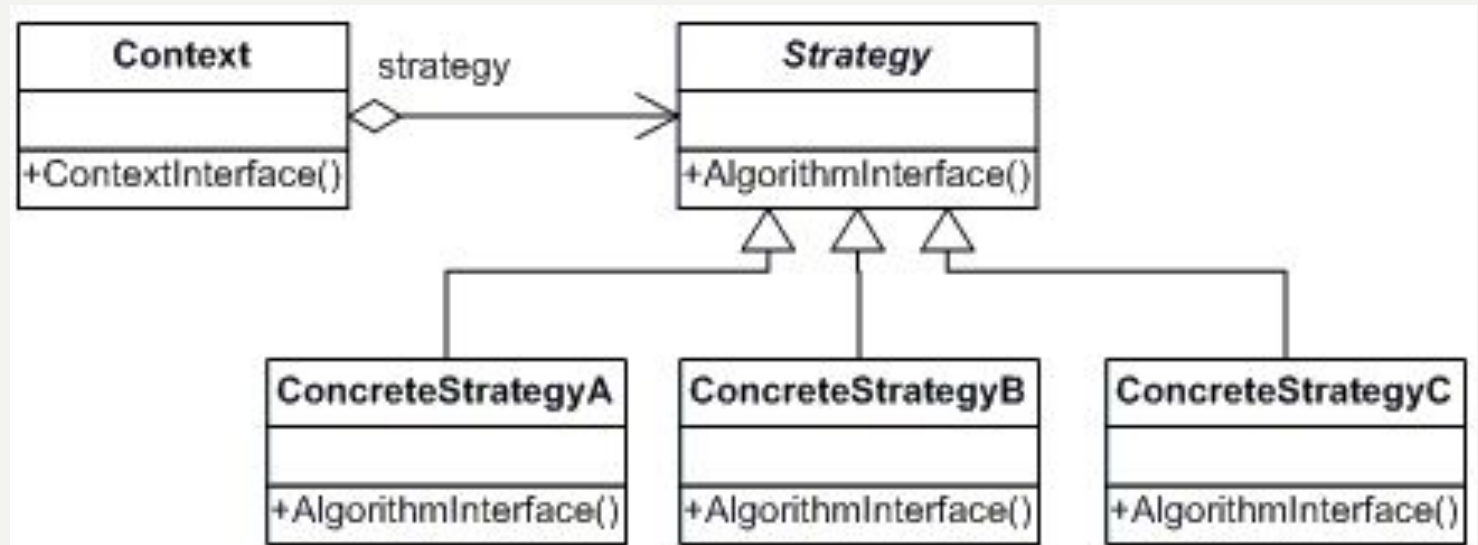
- Poseen diagramas estructurales similares
- Ambos se basan en la composición recursiva para organizar un numero abierto de objetos
- *Decorator* está diseñado para permitir agregar responsabilidades a objetos sin usar subclases, mientras que *Composite* hace foco en la representación en lugar del "embellecimiento".

A dark brown wavy line runs vertically along the left edge of the slide.A solid yellow vertical bar runs along the right edge of the slide.

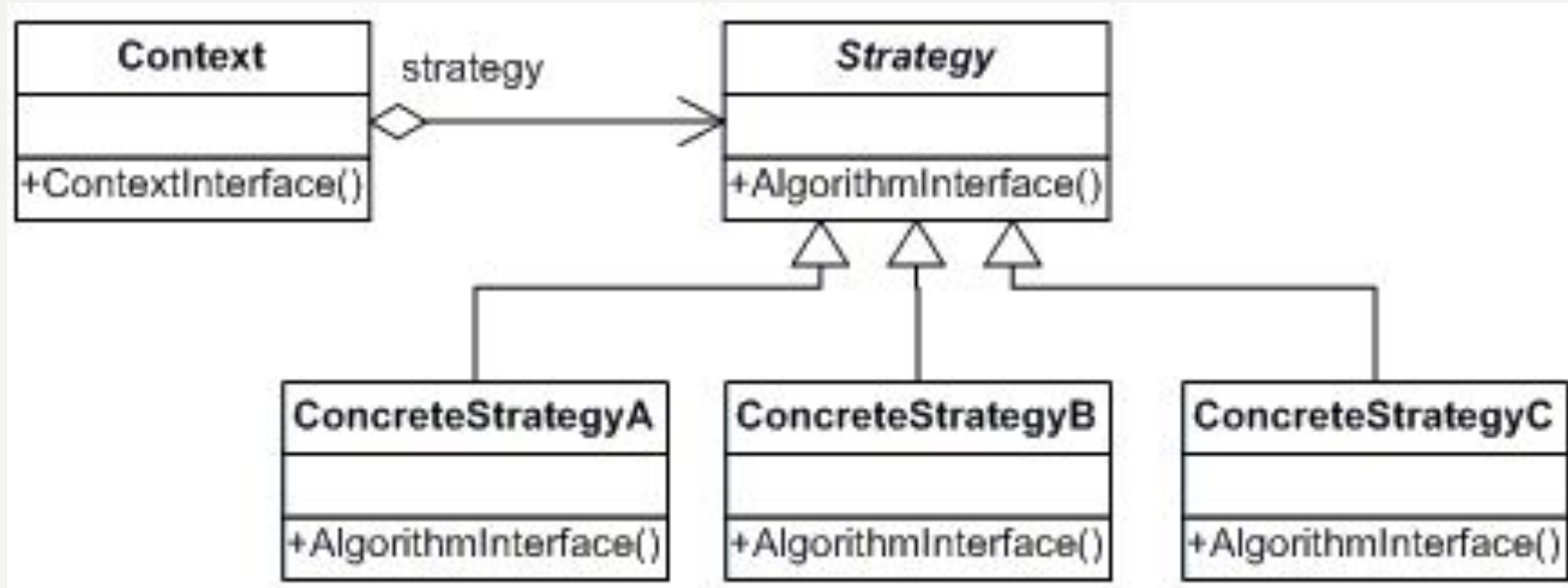
STRATEGY

STRATEGY (COMPORTAMIENTO)

Intención: Permite mantener un conjunto de algoritmos de entre los cuales el objeto cliente puede elegir aquel que le conviene e intercambiarlo dinámicamente según sus necesidades.



STRATEGY



STRATEGY

Participantes:

Strategy (SortStrategy)

Declara una interface común a todos los algoritmos soportados. El Contexto utiliza esta interface para llamar al algoritmo definido en una ConcreteStrategy

ConcreteStrategy (QuickSort, ShellSort, etc.)

Implementa el algoritmo usando la interface Strategy

Context (SortedList)

Se configura con un objeto ConcreteStrategy

Mantiene una referencia a un objeto Strategy

Puede definir una interface para que el Strategy acceda a sus datos

STRATEGY

Colaboraciones:

Strategy y Context interactúan para implementar el algoritmo. Un Context puede pasarle todos los datos necesarios al Strategy al invocar el algoritmo, o como alternativa puede pasarse a sí mismo como parámetro y que sea el mismo Strategy quien obtenga lo necesario del Context.

Un contexto delega los llamados de sus clientes en el strategy que almacena. Los clientes usualmente crearán y le asignarán un strategy a un contexto, y desde ese momento solo interactúan con el contexto.

STRATEGY

Aplicación (usar cuando):

- **Muchas clases relacionadas solo difieren en comportamiento. Strategy provee una forma de hacer ese comportamiento configurable.**
- **Se tienen muchas variantes de un mismo algoritmo. Se puede utilizar Strategy cuando esas variantes se implementan mediante jerarquía de clases.**
- **No se desea exponer estructuras de datos complejas y específicas de un algoritmo al cliente.**
- **Una clase define varios comportamientos y esto termina significando múltiples sentencias condicionales.**

STRATEGY

Consecuencias (+):

Familias de algoritmos relacionados. Utilizando jerarquía de clases puedo definir comportamiento común y reutilizarlo en las distintas ConcreteStrategy

Alternativa a subclasear, otra forma de lograr la implementación de múltiples algoritmos es

Subclasificando Context. Pero con eso estoy mezclando conceptos de entidades con implementaciones de algoritmos, dejo una estructura más difícil de entender, extender y mantener, y no puedo variar el comportamiento de un objeto dinámicamente.

El Strategy elimina evaluaciones condicionales, ya no necesito un switch para definir el comportamiento. Se lo pido al strategy asignado.

STRATEGY

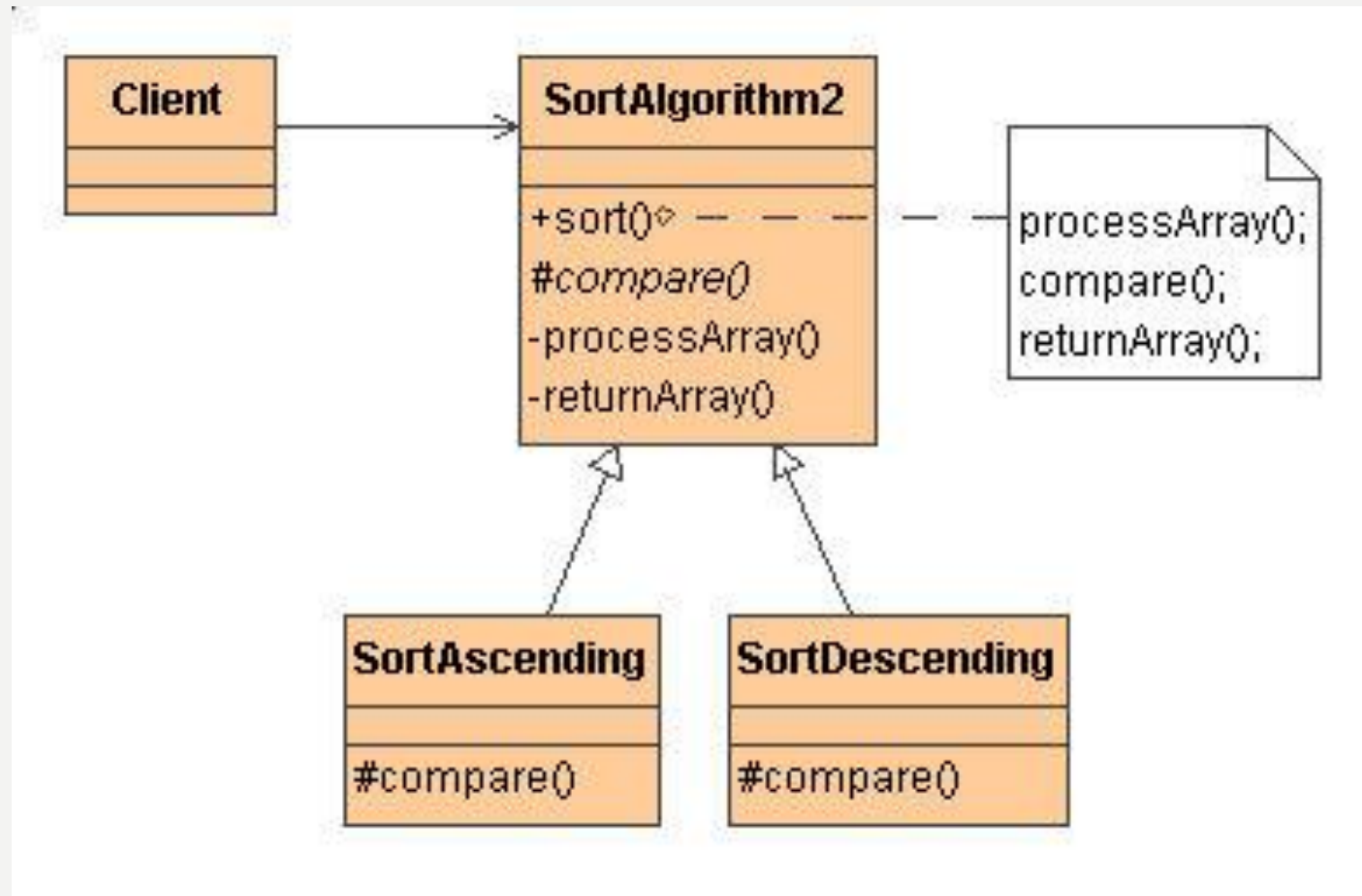
Consecuencias (-):

Los Clientes del Context deben estar al tanto de las distintas Strategies. Es el cliente quien debe seleccionar una estrategia adecuada para asignarla al contexto. Por lo tanto Strategy solo debe usarse cuando los cambios en el algoritmo implementado son relevantes para el Cliente.

Overhead en la comunicación Strategy-Context. Como todas las ConcreteStrategy deben implementar la misma interface Strategy, es común que los ConcreteStrategy mas simples no hagan uso de parte de la interface, lo cual quiere decir que van a recibir más información que la necesaria para su algoritmo concreto. Un paliativo es pasar el Context y que cada ConcreteStrategy se sirva lo que necesita

Aumento del número de objetos. Se crearán muchos objetos más que si no usáramos el Patrón. Si este es un problema se pueden implementar Strategies sin estado y compartirlos en toda la aplicación

STRATEGY



A dark brown, wavy vertical line runs along the left edge of the slide.A solid yellow vertical bar runs along the right edge of the slide.

FACADE

FACADE (ESTRUCTURAL)

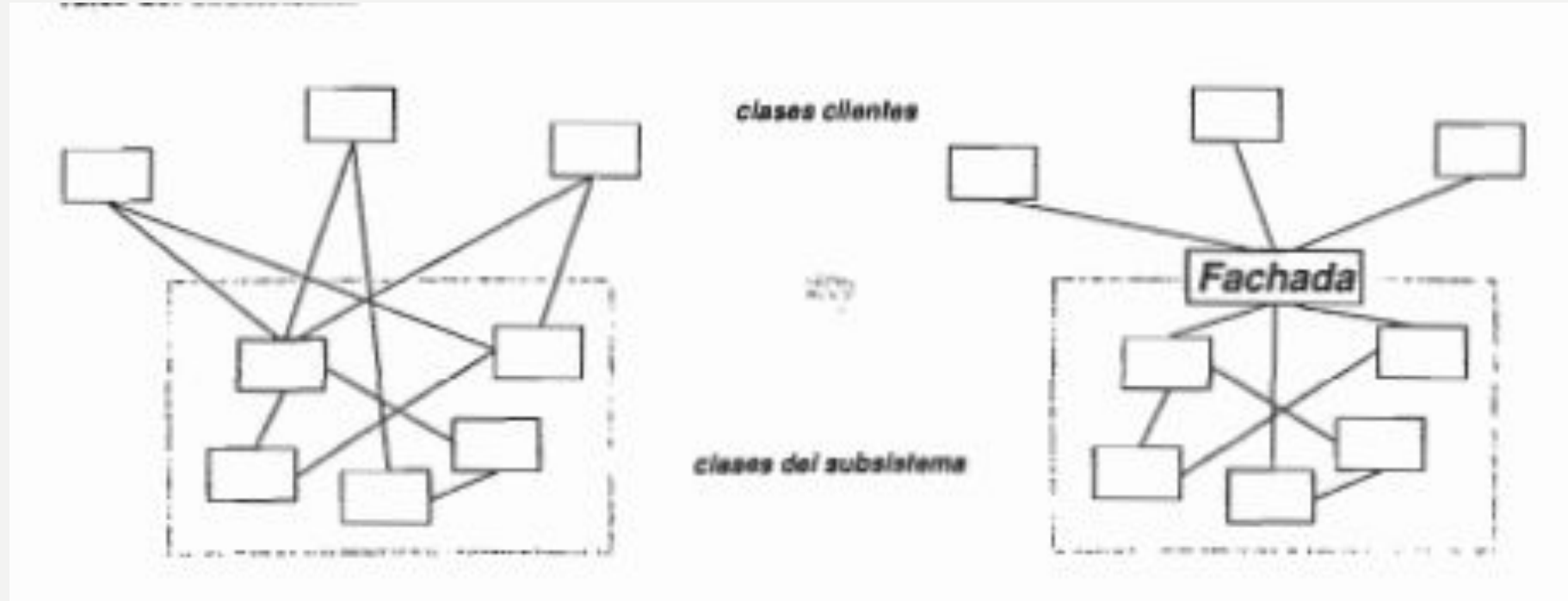
Intención: Proporciona una interfaz unificada para un conjunto de interfaces de un subsistema. Define una interfaz de alto nivel que hace que el subsistema sea más fácil de usar.

FACADE

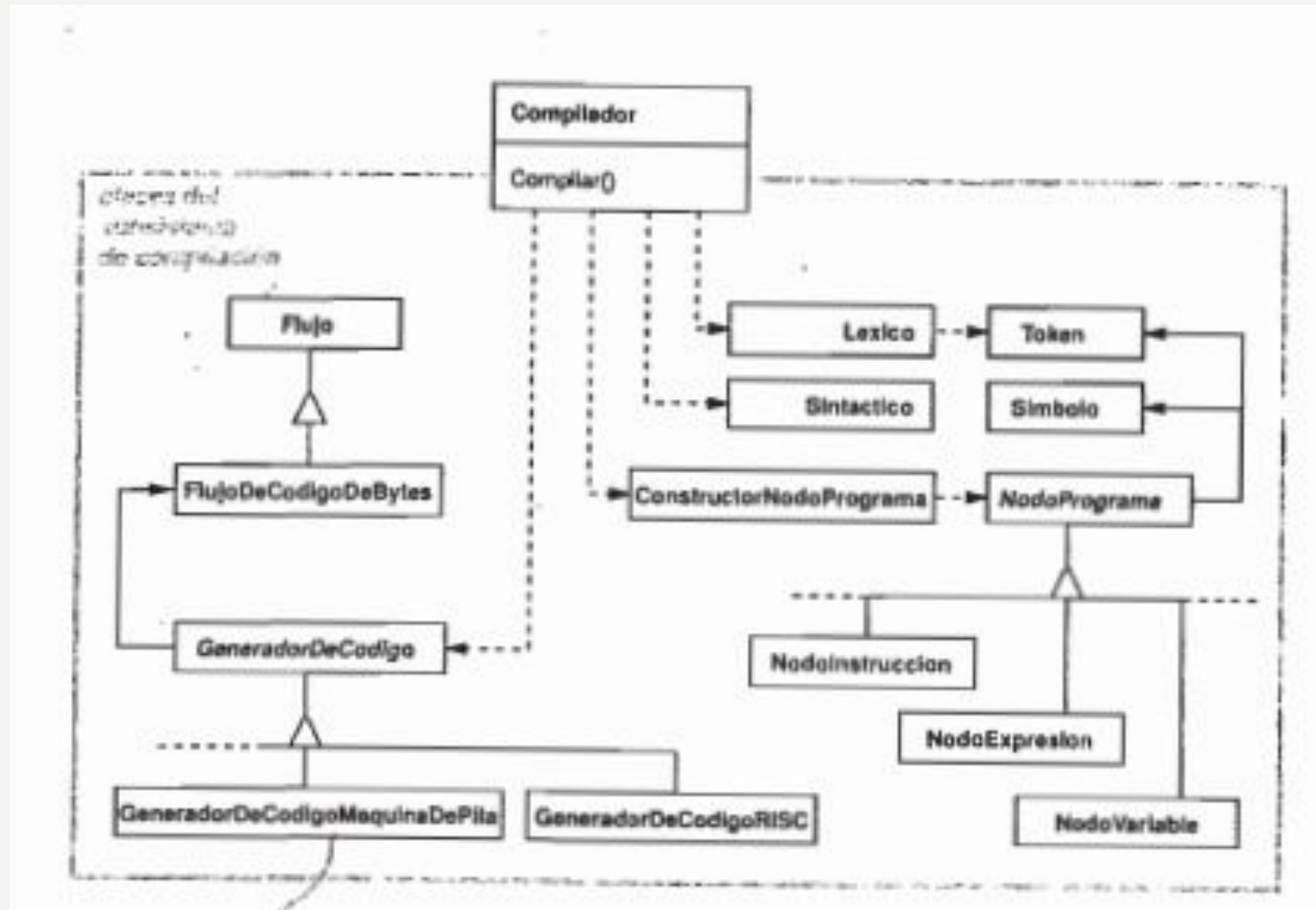
CONDICIONES DE USO:

- Se necesita una interfaz simple para proporcionar acceso a un sistema complejo.
- Hay muchas dependencias entre las implementaciones de sistemas y los clientes.

FACADE



FACADE - EJEMPLO



FACADE - EJEMPLO

Participantes:

- Fachada (Compilador):
 - Sabe que clases del subsistema son las responsables ante una petición.
 - Delega las peticiones de los clientes en los objetos apropiados del subsistema.
- Clases del subsistemas (Lexico, Sintactico, NodoPrograma, etc)
 - Implementan la funcionalidad del subsistema
 - Realizan los trabajos encomendados por el objeto Fachada.
 - No conocen a la fachada, es decir, no tienen relación con ella.

FACADE - EJEMPLO

Colaboración:

- Los clientes se comunican con el subsistema enviando peticiones al objeto Fachada, el cual las reenvía a los objetos apropiados del subsistema. Aunque los objetos del subsistema los que realizan el trabajo real, la fachada puede tener que hacer algo de trabajo para pasar de su interfaz a las del subsistema.
- Los clientes que usan la fachada no tienen que acceder directamente a los objetos del subsistema.

FACADE - EJEMPLO

Consecuencias:

- Oculta a los clientes los componentes del subsistema, reduciendo así el número de objetos con los que tratan los clientes y haciendo que el subsistema sea más fácil de usar.
- Promueve un débil acoplamiento entre el subsistema y sus clientes. Muchas veces los componentes de un subsistema están fuertemente acoplados. Un acoplamiento débil nos permite modificar los componentes del subsistema sin que sus clientes se vean afectados.
- No impide que las aplicaciones usen las clases del subsistema en caso de que sea necesario. De este modo, se puede elegir entre facilidad de uso y generalidad.



OBSERVER



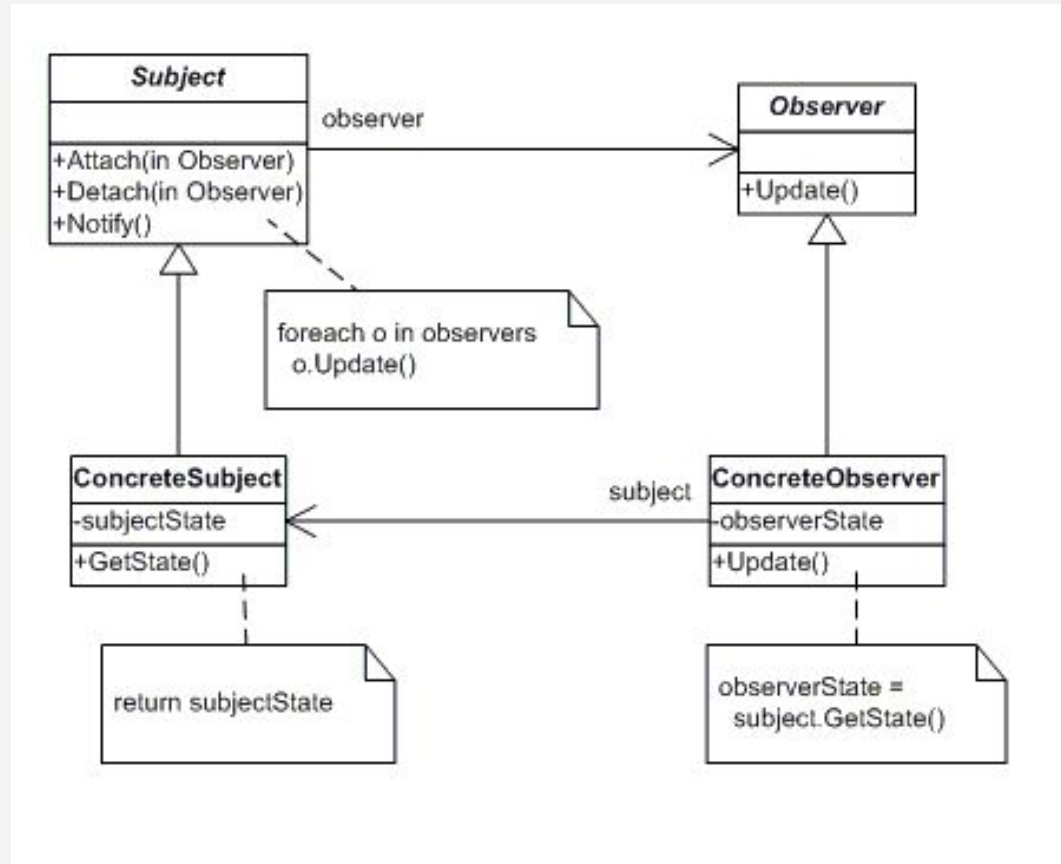
OBSERVER (COMPORTAMIENTO)

Intención: Define una dependencia del tipo uno-a-muchos entre objetos, de manera que cuando uno de los objetos cambia su estado, notifica este cambio a todos los dependientes.

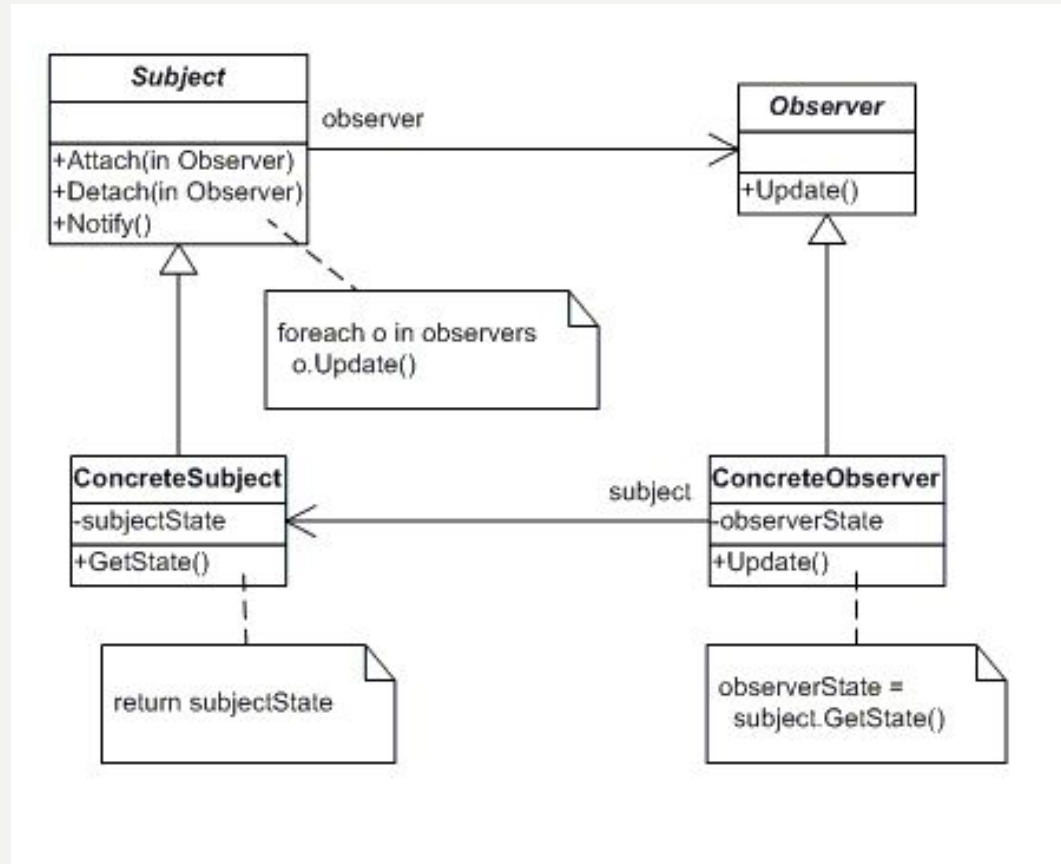
OBSERVER

- Se necesita consistencia entre clases relacionadas, pero con independencia.
- Los cambios de estado en uno o más objetos deben dar lugar a comportamiento en otros objetos

OBSERVER

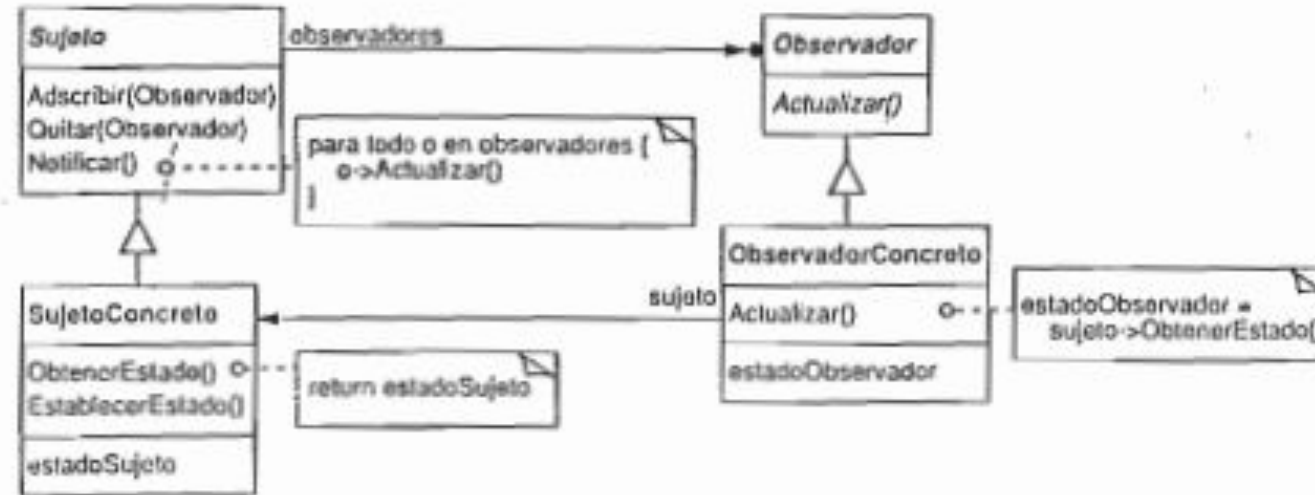


OBSERVER



OBSERVER - EJEMPLO

ESTRUCTURA



OBSERVER - EJEMPLO

PARTICIPANTES:

- **Sujeto:**
 - Conoce a sus observadores. Un sujeto puede ser observado por cualquier numero de objetos **Observador**.
 - Proporciona una interfaz para asignar o quitar objetos **Observador**
- **Observador:**
 - Define una interfaz para actualizar los objetos que deben ser notificados ante cambios de un sujeto
- **SujetoConcreto:**
 - Almacena el estado de interés para los objetos **ObservadorConcreto**.
 - Envía una notificación a sus observadores cuando cambia su estado.
- **ObservadorConcreto:**
 - Mantiene una referencia a un objeto **SujetoConcreto**
 - Guarda un estado que debería ser consistente con el del sujeto.
 - Implementa la interfaz de actualización del **Observador** para mantener su estado consistente con el del sujeto.